Swahili Tweet Sentiment Classification

TF-IDF with Logistic Regression, BiLSTM, AfriBERTa and AfroXLMR on the AfriSenti Swahili dataset

Project links

GitHub repository: https://github.com/Manzi453/Summative_Technique_I

Live system: https://summativetechniquei-x6jtcsn5dkzcft2gnskyjx.streamlit.app

Model on Hugging Face: https://huggingface.co/453ivan/swahili-sentiment-afrisenti

Demo video:
https://www.awesomescreenshot.com/video/57562961?key=2702f4389ab7cfd57daca6e03e6e940e

The live system runs on Streamlit Community Cloud. Free apps on that platform go to sleep after a period without visitors, so the first load can take a minute while the app wakes up and downloads the model (about 1.1 GB). The free tier gives an address of the form name.streamlit.app and does not support a custom domain. If the app is unavailable at marking time, the demo video shows it working and the README gives the commands to run it locally.

1. Abstract

Swahili is spoken by tens of millions of people in East and Central Africa, but labelled data for tasks such as sentiment analysis is still limited compared with English. This project builds and compares classifiers that label Swahili tweets as positive, neutral or negative, using the Swahili part of the AfriSenti dataset (3,011 tweets in the official training, development and test splits). The models are majority-class and random reference baselines, a TF-IDF with Logistic Regression baseline, a bidirectional LSTM trained from scratch (five seeds), and two pretrained African-language Transformers, AfriBERTa-base and AfroXLMR-base, fine-tuned for classification (two seeds each, at most four epochs, word embeddings frozen, because of an 8 GB laptop). Test-set differences are examined with bootstrap confidence intervals. AfroXLMR-base and AfriBERTa-base reach a macro-F1 of 0.562 and 0.558 (accuracy 0.577 and 0.608), against 0.483 for TF-IDF (accuracy 0.580) and 0.410 for the BiLSTM. Paired bootstrap intervals show that both Transformers score higher than TF-IDF and the BiLSTM, but they cannot separate the two Transformers. The gain comes mostly from the negative class, and accuracy barely exceeds the 0.594 obtained by always predicting neutral. The fine-tuned run with the best validation score (AfroXLMR-base) is served through a Streamlit web app.

2. Introduction

Sentiment analysis helps organisations and researchers follow public opinion on social media. For English there are many datasets and strong models. For African languages the situation is different: annotated corpora are rare, the text is often informal, and tweets mix languages, slang and spelling variants. AfriSenti (Muhammad et al., 2023) addresses part of this gap with tweets in 14 African languages labelled by native speakers, and it was the basis of SemEval-2023 Task 12 (Muhammad et al., 2023b). Swahili is a good language to study because the dataset has official splits, a published shared-task benchmark exists for

comparison, and pretrained models that saw Swahili during pretraining are available (Ogueji et al., 2021; Alabi et al., 2022).

Problem, users and justification. The task is to read one Swahili tweet and predict whether it is positive, neutral or negative. The intended users are people who need to follow public opinion in Swahili on social media, such as newsroom and civil-society analysts, public-sector communication teams in East Africa, and NLP researchers who need a documented reference system and evaluation protocol for Swahili. We did not interview such users; the use cases are plausible rather than validated. Swahili is a sensible choice because it is one of the three languages with the most speakers in Africa (Muhammad et al., 2023b) and because an annotated benchmark with official splits exists. General-purpose sentiment services support few African languages: Amazon Comprehend lists 12 languages and none is a sub-Saharan African language (Amazon Web Services, n.d.), and Google Cloud Natural Language lists 16 for sentiment analysis, not including Swahili (Google Cloud, n.d.). Microsoft's Azure AI Language does list Swahili among 94 language codes (Microsoft, 2026), so we do not claim that no established system handles Swahili sentiment; however, its documentation reports no accuracy for Swahili tweets, and we did not compare against it (Section 9). The contribution here is an open, reproducible comparison on a public benchmark, with an error analysis and a working application.

The objective of this project is to (1) prepare the Swahili AfriSenti data and document its characteristics, (2) compare a simple baseline, a recurrent network and two pretrained Transformers under one fixed split and one evaluation protocol, (3) analyse where the models fail and whether the differences between them are larger than random variation, and (4) deploy a working system that takes a Swahili tweet and returns a predicted sentiment. This is an individual assignment. Section 5 and the repository separate the existing resources used from the work implemented here.

Research questions. RQ1: Do pretrained African-language Transformers score higher than a strong lexical baseline and a recurrent network trained from scratch on the same split (Sections 6.1 and 6.2)? RQ2: Do the two Transformers differ from each other, and how much do results vary between seeds (Sections 6.2 and 6.3)? RQ3: Is the large share of unseen words a plausible reason for the Transformers' advantage (Section 7.3)? RQ4: Where do the models fail, and are some of those errors label problems (Section 7)?

3. Related Work

AfriSenti. Muhammad et al. (2023) released a Twitter sentiment benchmark for 14 African languages, annotated as positive, negative or neutral, and report experiments with classical and pretrained models. The data is released under a CC BY 4.0 licence on the project's GitHub repository. The Swahili subset is one of the languages used in SemEval-2023 Task 12 (Muhammad et al., 2023b). It follows NaijaSenti (Muhammad et al., 2022), a Twitter corpus for four Nigerian languages (Hausa, Igbo, Nigerian Pidgin and Yorùbá) in which language-specific models and language-adaptive fine-tuning worked best.

SemEval-2023 Task 12. The shared task Subtask A asked systems to classify tweets in each language into the three sentiment classes and used weighted F1 as the official metric. For Swahili, the baseline fine-tuning AfroXLMR-large scored a weighted F1 of 63.40 and the best submitted system scored 65.68 (Muhammad et al., 2023b). The best system (NLNDE) continued the pre-training of AfroXLMR-large on target-language text before fine-tuning, which improved clearly on fine-tuning it directly. These figures give the scale of what is

achievable: even large pretrained models reach only about two-thirds weighted F1 on Swahili tweets, so this is a hard task. We report weighted F1 next to macro-F1 so that our results can be compared with the shared task, while noting that our models, seeds and training budget differ.

Pretrained models for African languages. AfriBERTa (Ogueji et al., 2021) is a small multilingual model pretrained from scratch on under 1 GB of text from 11 African languages, Swahili included, and was shown to be competitive with larger multilingual models on text classification. AfroXLMR (Alabi et al., 2022) adapts the multilingual XLM-R model (Conneau et al., 2020) to African languages by multilingual adaptive fine-tuning on text from 17 African languages and three widely spoken high-resource languages. We fine-tune the base-sized versions of both, which are small enough to train on a laptop when the word embeddings are frozen (Section 5.4).

Simple baselines. Linear models over word and bigram features are strong baselines for text classification (Wang and Manning, 2012). On small datasets, where fine-tuning a large pretrained model has high variance between random seeds, a baseline can be hard to beat, which is why this project reports several seeds and confidence intervals rather than a single run.

Gap addressed. Our contribution is an empirical comparison under one fixed protocol with repeated runs, a test of whether the usual explanation for the pretrained models' advantage (unseen vocabulary) is supported by the data, an error analysis, and a deployed application. We do not claim a new model or a new state of the art.

4. Dataset and Data Preparation

4.1 Source and size

We use the Swahili files of AfriSenti Subtask A from the shared task repository (github.com/afrisenti-semeval/afrisent-semeval-2023): *sw_train.tsv*, *sw_dev.tsv* and *sw_test.tsv*, each with an identifier, the tweet text and a label (positive, negative or neutral). The official splits are used as they are; the development set serves as our validation set and the test set is used once per final model. In total there are 3,011 tweets.

4.2 Cleaning

Cleaning is deliberately light, because tweets are informal and the pretrained models have their own tokenizers. We lower-case the text, remove links, and collapse repeated whitespace. We do not remove stop words and we do not correct spelling. The released tweets contain no hashtags, mentions or emoji (Section 4.5), so these were not an issue, and no tweet became empty after cleaning. The same cleaning function is used in training and in the web app.

Table 1. Tweets per class in each split.

Split	Negative	Neutral	Positive	Total
Train	191 (10.6%)	1,072 (59.2%)	547 (30.2%)	1,810
Validation	48 (10.6%)	268 (59.2%)	137 (30.2%)	453
Test	80 (10.7%)	444 (59.4%)	224 (30.0%)	748

4.3 Characteristics

The three classes are imbalanced: neutral tweets are about 59% of every split and negative tweets only about 11%. A model that always predicts neutral therefore reaches an accuracy near 0.59 but a macro-F1 of only about 0.25, which is why macro-F1 is our primary metric and why we report this reference model. Tweets are short: the average is about 17 words and the median 16 or 17 in the three splits, and the 99th percentile of the training set is 41 words, so a maximum length of 128 subword tokens does not truncate meaningful content for almost all tweets. The training vocabulary has 9,098 distinct word forms in about 30,600 tokens, and 69.5% of the word forms occur only once. Because of this and the informal spelling, about 20% of the word tokens in the validation set and 21% in the test set never occur in the training set. This is the main reason we expect a bag-of-words model to struggle and pretrained subword models to help, an expectation we check in Section 6 rather than assume.

4.4 Overlap between splits

We checked for exact duplicates after cleaning. Nine tweets in the training set are duplicates of other training tweets, three validation tweets and one test tweet also occur in the training set. This overlap is too small to affect the results. The overlap flag is stored in the data files so it can be examined later.

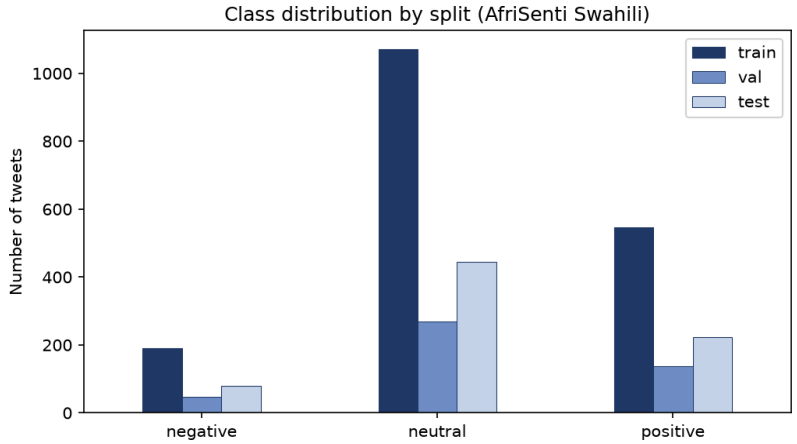


Figure 1. Tweets per class in each split: the neutral class dominates and the negative class is small.

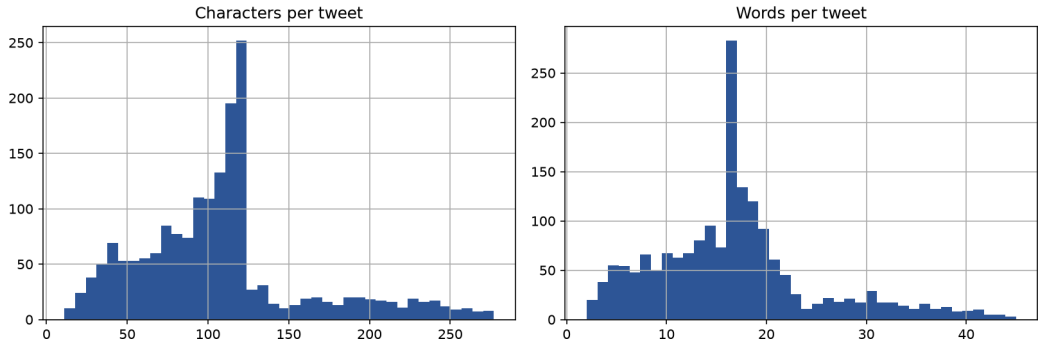


Figure 2. Tweet length in characters (left) and words (right) in the training set: tweets are short.

4.5 Data quality and limitations

According to the task paper, the tweets were labelled by native speakers and were collected with location-based and vocabulary-based heuristics (Muhammad et al., 2023b), so they are not a random sample of what Swahili speakers write. We have no annotator-agreement figures for the Swahili labels, so label noise cannot be quantified; Section 7.2 shows tweets whose labels look debatable. The released text is plain ASCII with no hashtags, mentions or emoji (they were removed in the release), and only 12 of the 3,011 tweets contain a link. Some tweets end in the middle of a word (for example sw_test_00334), which suggests truncation. Finally, our files differ in size from Table 1 of the task paper, which lists 1,198 / 454 / 749 Swahili tweets for train / dev / test, whereas the released files we downloaded contain 1,810 / 453 / 748. We used the files as released and did not investigate the difference.

5. Methodology

5.1 Reference models

Two models with no learning put the scores in context: one that always predicts the most frequent training class (neutral), and one that predicts classes at random with the training class frequencies.

5.2 Baseline: TF-IDF and Logistic Regression

Tweets are converted to TF-IDF vectors (Salton and Buckley, 1988) and classified with Logistic Regression using balanced class weights. We compared word 1 to 2-gram features (sublinear term frequency, up to 100,000 features), character n-gram features (2 to 5, within word boundaries, which helps with spelling variants; minimum document frequency 2, up to 200,000 features) and both combined, each with six values of the regularisation strength C (0.1, 0.5, 1, 5, 10 and 50), so 18 configurations in all. The vectorisers are fitted on the training set only. We chose the configuration with the best validation macro-F1: word and character features with C = 1 (validation macro-F1 0.4782).

5.3 BiLSTM

A bidirectional LSTM (Hochreiter and Schmidhuber, 1997; Schuster and Paliwal, 1997) in PyTorch: 100-dimensional word embeddings learned from scratch, a bidirectional LSTM layer with 64 hidden units per direction, mean and max pooling over the tweet's positions (joined into one vector), dropout 0.5 and a linear layer with three outputs. The vocabulary keeps the 2,777 words that occur at least twice in training (other words map to an unknown token) and sequences are cut at the 99th percentile of training lengths (41 words). Training uses Adam (learning rate 1e-3), batch size 32, class-weighted cross-entropy, gradient clipping at 1.0, at most 40 epochs and early stopping with patience 5 on validation macro-F1, on the CPU. Five random seeds were run. This model uses no pretraining, so it shows how far the 1,810 training tweets alone can go.

5.4 Fine-tuned pretrained models

We fine-tune AfriBERTa-base (castorini/afriberta_base; 8 Transformer layers, about 111 million parameters) and AfroXLMR-base (Davlan/afro-xlmr-base; 12 layers, about 278 million parameters, of which 192 million are the word-embedding table). Each tweet is split into subword tokens by the model's own tokenizer (median 25 tokens for AfriBERTa and 28 for

AfroXLMR, 99th percentile 64 and 66, no tweet is cut at the maximum length of 128, and no training token is unknown). The Transformer encoder turns the tokens into vectors, and the vector of the first token goes through a classification head (a dense layer and an output layer) that gives three scores; a softmax turns them into class probabilities. We train with AdamW (Loshchilov and Hutter, 2019), learning rate 3e-5 with a 10% linear warm-up and linear decay, weight decay 0.01, batch size 16, gradient clipping at 1.0, class-weighted cross-entropy and at most four epochs, with early stopping (patience 3) on validation macro-F1 and the best epoch's weights kept. Because the experiments ran on an 8 GB laptop (Apple M2, Apple GPU through PyTorch MPS, full precision), we made three compromises that we did not test against the full setting: the word-embedding table is frozen (it holds 49% of AfriBERTa's and 69% of AfroXLMR's weights, and freezing it roughly halved the time per step in a timing test), only two seeds were run for each model (seeds 1 and 2), and training was limited to four epochs. The deployed model is the single run with the best validation macro-F1 over all four fine-tuning runs (AfroXLMR-base, seed 1), so the choice never uses test data.

5.5 Evaluation protocol

The primary metric is macro-F1, the average of the three per-class F1 scores. We also report accuracy, weighted F1 (the shared-task metric) and macro precision and recall. To judge whether differences between models exceed sampling noise on a test set of 748 tweets, we compute 95% bootstrap confidence intervals (1,000 resamples of the test set) and paired bootstrap intervals for the difference between models (Berg-Kirkpatrick et al., 2012). A difference is called clear only if its interval excludes zero. All hyper-parameter and checkpoint choices use the validation set only.

5.6 Experimental setup and reproducibility

All experiments were run on one Apple M2 laptop with 8 GB of memory, with Python 3.14.7, PyTorch 2.14.1, Hugging Face Transformers 5.20.1, scikit-learn 1.9.1 and pandas 3.0.6 (requirements.txt lists the packages). Notebooks 01 to 04 in the repository, run in order, download the data, create the fixed split and produce the results files (notebook 04 must use SEEDS = [1, 2] and EPOCHS = 4, the settings of the reported results); scripts/extra_analysis.py computes, from the saved predictions, the paired bootstrap for every pair of models, the per-class scores, the unseen-vocabulary analysis and the error overlap. Random seeds are set for Python, NumPy and PyTorch, but training on the Apple GPU is not guaranteed to be repeatable bit for bit and the fine-tuning runs were not repeated, so a re-run may give slightly different numbers.

6. Experiments and Results

6.1 Main comparison

Table 2. Test-set results (748 tweets). Neural models: mean over seeds (five for the BiLSTM, two for each fine-tuned Transformer), standard deviation in brackets for macro-F1. Train time is the mean per run on the laptop; the BiLSTM ran on the CPU and the Transformers on the Apple

GPU, and the AfroXLMR mean includes one run of 1,978 s (the other took 796 s), probably because the laptop was short of memory during the first.

Model	Accuracy	Macro-F 1 (std)	Weighted F1	Train time
Majority class (always neutral)	0.5936	0.2483	0.4422	none
Random by class frequency	0.4358	0.3213	0.4357	none
TF-IDF + Logistic Regression	0.5802	0.4829	0.5737	0.6 s
BiLSTM (5 seeds)	0.4749	0.4102 (0.0505)	0.4888	19.7 s
AfriBERTa-base fine-tuned (2 seeds)	0.6083	0.5578 (0.0087)	0.6067	369 s
AfroXLMR-base fine-tuned (2 seeds)	0.5769	0.5617 (0.0182)	0.5825	1,387 s

By macro-F1 the ranking is AfroXLMR-base (0.562) and AfriBERTa-base (0.558), then TF-IDF (0.483), the BiLSTM (0.410), the random reference (0.321) and the majority class (0.248). Both Transformers improve on the TF-IDF baseline by about 0.075 and 0.079 macro-F1, whereas the BiLSTM trained from scratch is below TF-IDF on average. Accuracy tells a weaker story: AfriBERTa reaches 0.608, only 0.015 above the 0.594 of always answering neutral, and AfroXLMR (0.577) is below it, because the Transformers' gains are on the minority classes, not on the dominant neutral class (Section 7.1). Weighted F1, the shared-task metric, is 60.7 for AfriBERTa and 58.3 for AfroXLMR, below the 63.40 of the shared-task baseline and the 65.68 of the best Swahili system (Muhammad et al., 2023b). This is not a like-for-like comparison: we fine-tune base-size models with frozen word embeddings for at most four epochs, with two seeds and no hyper-parameter search.

6.2 Are the differences real?

Table 3. Test macro-F1 with 95% bootstrap confidence intervals (best-validation run of each model).

Model	Macro-F1	CI low	CI high
TF-IDF + Logistic Regression	0.4829	0.4393	0.5263
BiLSTM	0.4664	0.4289	0.5045
AfriBERTa-base	0.5517	0.5088	0.5889
AfroXLMR-base	0.5488	0.5085	0.5873

The confidence intervals in Table 3 overlap, so we compare models with paired bootstrap differences (1,000 resamples of the same test tweets; Table 3b). Both Transformers score higher than TF-IDF (AfriBERTa +0.068 [0.014, 0.119], AfroXLMR +0.066 [0.013, 0.115]) and higher than the BiLSTM (AfriBERTa +0.085 [0.036, 0.135], AfroXLMR +0.082 [0.038, 0.128]); all four intervals exclude zero. The lower ends are small, though (0.014 and 0.013 against TF-IDF), so the size of the advantage over TF-IDF is uncertain. The data do not show a clear difference between the two Transformers (AfriBERTa minus AfroXLMR +0.003 [−0.043, 0.045]) or between TF-IDF and the BiLSTM (TF-IDF minus BiLSTM +0.017 [−0.029, 0.058]), so we do not claim that one Transformer is better or that TF-IDF beats this BiLSTM run. Two cautions apply. The notebook's reference for paired comparisons is the model with the

highest test macro-F1 (AfriBERTa-base); this choice was made after seeing test scores, so its intervals are slightly favourable to it, whereas the deployed model was chosen on validation data. And the intervals reflect only the sampling of the test tweets, not the randomness of training, because each model is represented by one run.

Table 3b. Paired bootstrap difference in test macro-F1 (A minus B, where A is the model with the higher test macro-F1) for every pair of models, with 95% intervals (results/metrics/bootstrap_paired_all.csv).

Comparison (A vs B)	A minus B	CI low	CI high
AfriBERTa-base vs TF-IDF	+0.068	+0.014	+0.119
AfroXLMR-base vs TF-IDF	+0.066	+0.013	+0.115
AfriBERTa-base vs BiLSTM	+0.085	+0.036	+0.135
AfroXLMR-base vs BiLSTM	+0.082	+0.038	+0.128
AfriBERTa-base vs AfroXLMR-base	+0.003	−0.043	+0.045
TF-IDF vs BiLSTM	+0.017	−0.029	+0.058

6.3 Variation across seeds

Over the five BiLSTM seeds macro-F1 ranges from 0.336 to 0.466 (standard deviation 0.051), and the epoch with the best validation score ranges from 1 to 16, so the model trained from scratch is unstable: its worst seed is close to the random reference (0.321). The fine-tuned models are more stable, but we ran only two seeds each: AfriBERTa 0.552 and 0.564, AfroXLMR 0.549 and 0.575. Their ranges overlap, which agrees with the paired bootstrap, and every fine-tuned run (0.549 to 0.575) is above every BiLSTM run and above TF-IDF (0.483). With two seeds the standard deviations (0.009 and 0.018) are only rough guides. The training curves (Figure 3) show different behaviour for the two models in seed 1. For AfriBERTa the training loss falls from about 0.96 to 0.35 over four epochs while validation macro-F1 peaks at epoch 1 (0.522) and then stays about 0.01 lower, so most of the later training did not help. For AfroXLMR the validation macro-F1 rises from about 0.47 to 0.546 and is still rising at epoch 4, the last epoch we allowed, so this model may be under-trained and a longer budget could change the ranking between the two Transformers. We did not test this.

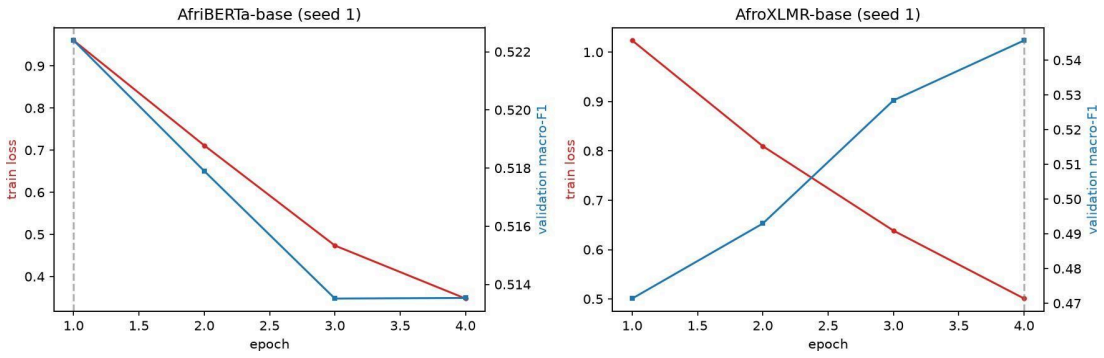


Figure 3. Training loss (red) and validation macro-F1 (blue) per epoch for the first fine-tuning run (seed 1) of each Transformer; the dashed line marks the best epoch. The BiLSTM curve is in figures/bilstm/fig_training_curve.png.

6.4 Experiment overview

Table 4. Configurations tried and validation scores.

Configuration	What changed	Validation macro-F1
TF-IDF word 1-2-gram	C in {0.1, 0.5, 1, 5, 10, 50}; best C = 10	0.4367
TF-IDF character 2-5-gram	C in {0.1, 0.5, 1, 5, 10, 50}; best C = 0.1	0.4772
TF-IDF word + character	C in {0.1, 0.5, 1, 5, 10, 50}; best C = 1 (chosen)	0.4782
BiLSTM	5 seeds, same settings (mean of best validation scores)	0.3833
AfriBERTa-base	2 seeds, same settings (mean of best validation scores)	0.5155
AfroXLMR-base	2 seeds, same settings (mean of best validation scores)	0.5397

Table 4 lists every configuration compared. For the baseline, character n-grams and word-plus-character features give higher validation macro-F1 (0.477 and 0.478) than word features alone (0.437), which fits our reason for adding character n-grams (Swahili word endings and spelling variants), although we did not test whether differences of a few hundredths on 453 validation tweets are reliable. The neural rows are means over seeds of each run's best validation macro-F1. They are lower than the test scores in Table 2 for every neural model (for example 0.516 against 0.558 for AfriBERTa), so the validation and test sets are not equally hard, which is another reason not to read small differences literally. Experiments were therefore of two kinds: the feature type of the baseline, and the model family (a recurrent network trained from scratch against pretrained Transformers) with the same cleaned text, fixed split, class-weighted loss and early-stopping rule. We did not search learning rates, sequence lengths or epochs for the neural models, and we did not run ablations of the frozen embeddings or of the class weights, which is a limitation (Section 9).

6.5 Discussion

The clearest result is the ordering of the systems: pretrained African-language Transformers above TF-IDF, TF-IDF above a BiLSTM trained from scratch, and all of them above the reference models, with the Transformers' gain concentrated in the negative class (Section 7.1). This agrees in direction with the shared task, where the strongest Swahili systems used large pretrained African-language models and the winning team improved on plain fine-tuning by continuing pre-training on target-language text (Muhammad et al., 2023b), and with NaijaSenti, where language-specific models and language-adaptive fine-tuning worked best (Muhammad et al., 2022). Our scores are lower than the shared-task Swahili figures (weighted F1 58 to 61 against 63.40 and 65.68), which would fit the smaller models and the small training budget used here, but we did not test that explanation. Two results are less expected. The BiLSTM was not better than TF-IDF even though TF-IDF ignores word order (its five-seed mean is 0.073 lower and its best seed 0.017 lower), which is consistent with the finding that linear models over word and bigram features are strong baselines on small datasets (Wang and Manning, 2012). And the two Transformers could not be separated although AfroXLMR-base has more than twice the parameters of AfriBERTa-base (about 278

against 111 million), which is consistent with the report that the small AfriBERTa is competitive with larger multilingual models (Ogueji et al., 2021). Neither explanation was tested here.

7. Error Analysis and Discussion

7.1 Per-class performance

Table 5. Per-class F1 on the test set (support in the second column).

Model	Support	Negative F1	Neutral F1	Positive F1
TF-IDF + Logistic Regression	80 / 444 / 224	0.305	0.675	0.468
BiLSTM (best-validation run, seed 3)	80 / 444 / 224	0.300	0.654	0.445
AfriBERTa-base (best-validation run, seed 1)	80 / 444 / 224	0.506	0.694	0.455
AfroXLMR-base (best-validation run, seed 1; deployed)	80 / 444 / 224	0.535	0.604	0.507

The negative class (80 test tweets) is the hardest for TF-IDF and the BiLSTM (F1 0.305 and 0.300). TF-IDF finds only 25% of the negative tweets despite balanced class weights, whereas the BiLSTM finds 45% of them but with a precision of only 0.225. The Transformers' advantage is concentrated in this class: negative F1 rises to 0.506 (AfriBERTa) and 0.535 (AfroXLMR), about 0.2 above TF-IDF, while neutral and positive F1 move by less than 0.08 in either direction. With only 80 negative tweets these per-class scores are noisy. Among the fine-tuned models the hardest class is positive for AfriBERTa (0.455) and neutral for AfroXLMR (0.604), so our initial expectation that negative would be hardest holds for the baseline and the BiLSTM but not for the fine-tuned models. AfroXLMR over-predicts the sentiment classes (neutral recall 0.53; 156 neutral tweets predicted positive is the largest single confusion), whereas AfriBERTa is more conservative (neutral recall 0.72, but 117 positive tweets predicted neutral). Across all four models, 85% to 95% of the errors involve the neutral class (for AfroXLMR, 309 of 328; results/metrics/error_overlap.csv), and confusions between positive and negative are rare. The main difficulty is deciding whether a tweet expresses sentiment at all, not which polarity it has. Normalised confusion matrices are in Figure 4.

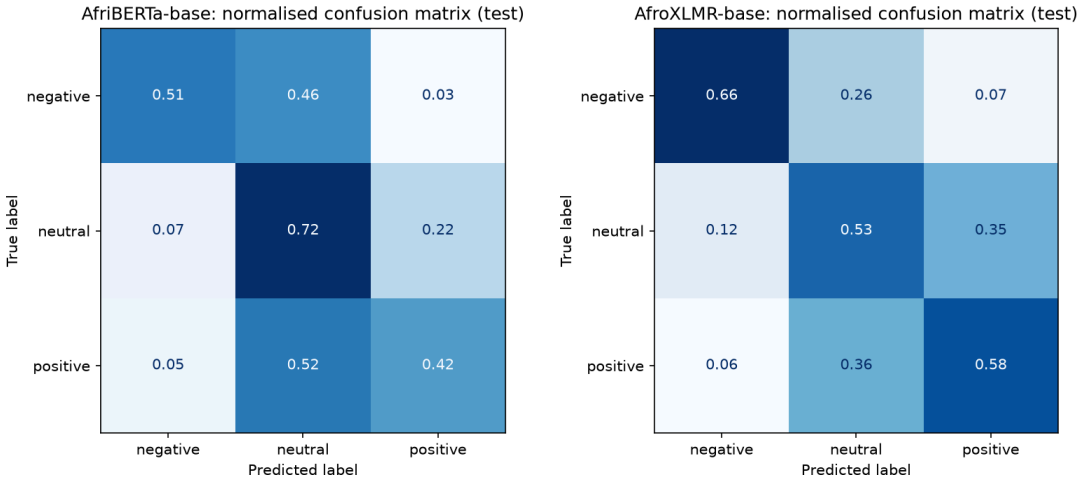


Figure 4. Normalised confusion matrices on the test set for the best-validation run of AfriBERTa-base (left) and AfroXLMR-base (right).

7.2 Where the models fail

Table 6. Six test tweets that all four models misclassify, with the same wrong prediction from every model. English glosses are approximate, machine-assisted translations that have not been checked by a native Swahili speaker.

Tweet (Swahili)	True	Predicted	Likely reason
Kwa sasa ni tawi lingine tu la ccm kama mengine period wanafki tu *[sw_test_00281] Gloss: For now it is just another branch of CCM (a Tanzanian political party) like the others, period, just hypocrites.*	negative	neutral	The negativity rests on one insult ('wanafiki', spelt 'wanafki' here, so probably an unseen word) and on knowing what CCM is; no common sentiment word appears.
Kuna mikoa wahudumu wamefanya tip ni haki yao ya msingi tena kwa huduma mbovuuuu tupate tuu katiba mpya aisee *[sw_test_00531] Gloss: In some regions waiters have made tips their basic right, even for bad service; let's just get a new constitution.*	negative	neutral	The complaint uses an elongated spelling ('mbovuuuu' for 'mbovu', bad) that probably splits into unfamiliar word pieces, and the tweet mixes two topics.
Wizara kushirikiana na mejimenti ya Utumishi imefanikisha zoezi la kuwasajili na kuwahakiki watumishi wa umma 507026 *[sw_test_00096] Gloss: The Ministry, working with the Public Service management, has succeeded in registering and verifying public servants (507026).*	neutral	positive	A factual announcement labelled neutral that contains success vocabulary ('imefanikisha'); the models seem to follow the polarity word, the label follows the news-report genre.
Ahsante kwa kutembelea ukurasa wetu Tumia Ezypesa ushinde *[sw_test_00110] Gloss: Thank you for visiting our page. Use Ezypesa and win.*	neutral	positive	Advertising text that opens with a thank-you formula (the TF-IDF model's top positive words include 'asante'); politeness is not an opinion.

Tweet (Swahili)	True	Predicted	Likely reason
Watu wengine hawatakupenda hata ufanye nini na watu wengine hawataacha kukupenda hata ufanye nini Nenda mahali penye upendo *[sw_test_00005] Gloss: Some people will not love you whatever you do, and others will not stop loving you whatever you do. Go where there is love.*	positive	neutral	Inspirational advice with implicit sentiment: negated verbs ('hawatakupenda') next to 'upendo' (love) give mixed signals; a reader could also call it neutral or mixed.
Tafadhali tunaomba kufahamu unapata changamoto ipi kwa msaada zaidi gt BM *[sw_test_00123] Gloss: Please, we would like to know which problem you are having, for more help.*	positive	neutral	A customer-service reply with no opinion that is labelled positive; this may be a label disagreement rather than a model error.

We selected these six from the 111 test tweets (14.8%) that all four models get wrong, to show different kinds of error; 207 tweets (27.7%) are correct in all four models. The table is therefore not a random sample and does not show how common each kind of error is. Four of the six (00281, 00531, 00096, 00110) look like genuine model errors with a visible cause in the wording. For the other two (00005, 00123) we think the label itself is debatable, so some of the errors may be disagreements about the label rather than model mistakes; we could not measure how many, because we have no annotator-agreement data. These readings are hypotheses, were not tested, and should be checked by a native speaker.

7.3 Why did the results come out this way?

The statements below are candidate explanations for the results. Each is marked as tested (with the evidence) or untested.

- **Unseen vocabulary.** About 20% of validation and 21% of test word tokens never occur in training (Section 4.3), and 93% of test tweets contain at least one such word. A bag-of-words model cannot use them, whereas subword models can, so pretrained models should help most when many words are unseen. **Status: partly tested, inconclusive.** We grouped the test tweets by the share of their words that are unseen (results/metrics/unseen_vocab_analysis.csv). Among the 201 tweets with at least 30% unseen words, AfriBERTa's accuracy is 0.064 higher than TF-IDF's (95% bootstrap interval [0.005, 0.129]); among the 159 tweets with at most 10% unseen words it is 0.026 lower ([−0.113, 0.063]). That fits the explanation, but the difference between the two gaps (+0.090) has an interval of [−0.017, 0.196] that includes zero, and AfroXLMR shows no such pattern (−0.005 against −0.029; the difference between the gaps is +0.024, [−0.102, 0.147]). The data neither confirm nor reject this explanation.
- **Small training set.** With 1,810 tweets, a BiLSTM from scratch has little to learn from, and fine-tuning may vary between seeds. **Status: partly tested.** The BiLSTM's macro-F1 ranges from 0.336 to 0.466 over five seeds (Section 6.3), which shows high variance. The fine-tuned models have only two seeds, so their variance is poorly estimated. We did not test whether the BiLSTM is limited by data size, for example with a learning curve, so that part is untested.
- **Class imbalance.** Only 191 negative training tweets, and class weights were used in every learned model. **Status: partly tested.** Negative F1 is the lowest for TF-IDF and the BiLSTM (about 0.30), and the Transformers gain mostly in this class (Section 7.1), so the

imbalance matters for the weaker models. We did not train without class weights or with other remedies such as oversampling, so the effect of the weights themselves is untested.

- **Label subjectivity and sarcasm.** Plausible given the shared-task scores, but not measured here. **Status: untested,** supported only by our reading of six errors, two of which (00005, 00123) have debatable labels (Section 7.2). We had no annotator-agreement figures.
- **The neutral boundary.** Most errors are about whether a tweet expresses sentiment at all: 85% to 95% of each model's errors involve the neutral class. **Status: tested descriptively,** from the confusion matrices and results/metrics/error_overlap.csv. Why this happens (announcements and advertisements that contain polarity words are labelled neutral, Table 6) is a hypothesis from six tweets and is untested.
- **Limited training budget and frozen embeddings.** AfroXLMR's seed-1 run was still improving at the last allowed epoch (Section 6.3), and the word embeddings were frozen to fit the laptop. **Status: untested.** We did not compare with longer training or with unfrozen embeddings, so we cannot say how much these choices cost.

8. Deployment

The system is a Streamlit web app (streamlit_app/streamlit_app.py) with three tabs: Classify, Model results and About. The user pastes a Swahili tweet, or picks one of six real test tweets as an example, and presses Classify; the app shows the predicted sentiment, the probability of each class, a note when the top probability is below 0.5, and, for an example, the human label. The second tab shows the test results of all models, generated from results/metrics_summary.csv, and the third describes the data, the limits and privacy: the app code does not save what the user types.

How the app uses the model. The app loads the fine-tuned model with the Hugging Face Transformers library and PyTorch on the CPU. It cleans the input with the same function as in training (lower-case, remove links, collapse spaces), tokenises it with the model's own tokenizer (at most 128 tokens), runs one forward pass and applies a softmax to the three outputs to get class probabilities. The model files (about 1.1 GB: weights, tokenizer and configuration) are not stored in GitHub. The app reads them from the local folder models/best_sentiment when it exists, and otherwise from the Hugging Face repository recorded in results/metrics/deployed_model.json (453ivan/swahili-sentiment-afrisenti). The deployed run is AfroXLMR-base, seed 1, chosen because it had the best validation macro-F1 of all fine-tuning runs (0.546); its test macro-F1 is 0.549 and its accuracy 0.561 (Table 3 and results/metrics/finetune_runs.csv). We did not compress or quantise the model, so the memory limit of the free host is a risk (Streamlit's documentation gives a range of up to about 2.7 GB per app, and older sources say 1 GB). The repository holds the AfroXLMR-base weights, tokenizer and configuration saved by notebook 04 (about 1.1 GB), and the deployed app reads the model from it.

The app is hosted on Streamlit Community Cloud at a free streamlit.app address. A custom domain would need a paid host and was not used.

9. Limitations and Future Work

- **Small data and one split.** The test set has 748 tweets and only 80 negative ones, so scores for the negative class are noisy. Bootstrap intervals show this uncertainty but do not remove it. Cross-validation or a larger dataset would help.
- **Limited tuning and training budget.** We used one setting per neural model and tuned only the baseline's regularisation. Fine-tuning used two seeds, at most four epochs and frozen word embeddings because of an 8 GB laptop; one AfroXLMR run was still improving at the last epoch, and the effects of freezing and of the epoch limit were not tested. A search over learning rates, sequence lengths, longer training and larger models (for example AfroXLMR-large, used by the shared-task baseline) could change the ranking.
- **Label subjectivity.** Sentiment in tweets is subjective, and sarcasm, code-switching and mixed feelings are hard. The labels are not guaranteed to be consistent.
- **Provenance of pretrained models.** The pretraining data of AfriBERTa and AfroXLMR may overlap with public tweets or web text, and we could not check whether any test tweets were seen in pretraining.
- **Domain.** The app was trained on tweets from a limited period and set of topics. It is likely to be less reliable on formal text and always returns one of three labels, even for text with no sentiment.
- **Error analysis depth.** The qualitative analysis covers a small, hand-selected sample of errors, and the glosses and readings have not been checked by a native Swahili speaker.
- **Hosting.** The free host puts the app to sleep when idle, has limited memory for a 1.1 GB model, and does not support a custom domain.
- **No comparison with commercial services.** Azure AI Language lists Swahili among its supported languages (Microsoft, 2026). We did not evaluate it on this test set, so we cannot say how our models compare with it.

10. Conclusion

We built and compared a TF-IDF with Logistic Regression baseline, a BiLSTM and two fine-tuned African-language Transformers on the Swahili part of AfriSenti, under the official split and one protocol. Both Transformers reach a macro-F1 of about 0.56 (AfriBERTa 0.558 ± 0.009, AfroXLMR 0.562 ± 0.018 over two seeds), against 0.483 for TF-IDF and 0.410 ± 0.051 for the BiLSTM. Paired bootstrap intervals show that both Transformers score higher than TF-IDF and the BiLSTM on the test set (differences of 0.066 to 0.085 macro-F1, intervals excluding zero), but not that one Transformer is better than the other. The gain comes mostly from the negative class; accuracy (0.577 to 0.608) barely exceeds the 0.594 of always predicting neutral, and weighted F1 (58 to 61) is below the 63 to 66 reported in the shared task for larger models and bigger budgets. The fine-tuned AfroXLMR run with the best validation score is served by a Streamlit web app.

Four lessons stand out. First, reporting several seeds and intervals limited what we could claim: the BiLSTM's seeds range from near-random (0.336) to just below TF-IDF (0.466), and the two Transformers cannot be ranked. Second, macro-F1 and accuracy tell different stories on imbalanced data, and most errors (85% to 95%) sit on the boundary between neutral and sentiment-bearing tweets, which is also where labels look subjective. Third, the unseen-vocabulary explanation is plausible for AfriBERTa but inconclusive. Fourth, practical

limits shaped the work: an 8 GB laptop led to frozen embeddings, two seeds and four epochs, and the free host limits what can be deployed. Future work is to run more seeds and longer training with unfrozen embeddings, to tune and compare a larger model, to test the class-weight and label-noise explanations, to have a native speaker review the errors and labels, and to compare with commercial APIs.

Existing resources and own work

Existing resources used: the AfriSenti dataset (Muhammad et al., 2023); the pretrained models castorini/afriberta_base and Davlan/afro-xlmr-base; the libraries scikit-learn, PyTorch, Hugging Face Transformers and Tokenizers, pandas, NumPy, matplotlib and Streamlit. **Implemented in this project:** the data loading, cleaning and checks, the reference and TF-IDF baselines, the BiLSTM, the fine-tuning pipeline, evaluation with bootstrap intervals, the supplementary analyses (scripts/extra_analysis.py), the error analysis and the Streamlit web app.

11. References

Alabi, J. O., Adelani, D. I., Mosbach, M., and Klakow, D. (2022). Adapting pre-trained language models to African languages via multilingual adaptive fine-tuning. *Proceedings of COLING 2022*. https://aclanthology.org/2022.coling-1.382

Amazon Web Services. (n.d.). Languages supported in Amazon Comprehend. Amazon Comprehend Developer Guide. https://docs.aws.amazon.com/comprehend/latest/dg/supported-languages.html (accessed 18 October 2026).

Berg-Kirkpatrick, T., Burkett, D., and Klein, D. (2012). An empirical investigation of statistical significance in NLP. *Proceedings of EMNLP-CoNLL 2012*.

Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzman, F., et al. (2020). Unsupervised cross-lingual representation learning at scale. *Proceedings of ACL 2020*. https://aclanthology.org/2020.acl-main.747

Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*. https://doi.org/10.18653/v1/N19-1423

Google Cloud. (n.d.). Natural Language API: Supported languages. https://docs.cloud.google.com/natural-language/docs/languages (accessed 18 October 2026).

Hochreiter, S., and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation, 9*(8), 1735-1780.

Loshchilov, I., and Hutter, F. (2019). Decoupled weight decay regularization. *ICLR 2019*. arXiv:1711.05101.

Microsoft. (2026). Sentiment Analysis and Opinion Mining language support. Azure AI Language documentation. https://learn.microsoft.com/en-us/azure/ai-services/language-service/sentiment-opinion-mining/language-support (accessed 18 October 2026).

Muhammad, S. H., Adelani, D. I., Ruder, S., Ahmad, I. S., Abdulmumin, I., Bello, B. S., et al. (2022). NaijaSenti: A Nigerian Twitter sentiment corpus for multilingual sentiment analysis. *Proceedings of LREC 2022*, 590-602. https://aclanthology.org/2022.lrec-1.63

Muhammad, S. H., Abdulmumin, I., Ayele, A. A., Ousidhoum, N., Adelani, D. I., Yimam, S. M., et al. (2023). AfriSenti: A Twitter sentiment analysis benchmark for African languages. *Proceedings of EMNLP 2023*. https://aclanthology.org/2023.emnlp-main.862

Muhammad, S. H., Abdulmumin, I., Yimam, S. M., Adelani, D. I., Ahmad, I. S., Ousidhoum, N., et al. (2023b). SemEval-2023 Task 12: Sentiment analysis for African languages (AfriSenti-SemEval). *Proceedings of SemEval-2023*. arXiv:2304.06845. https://arxiv.org/abs/2304.06845

Ogueji, K., Zhu, Y., and Lin, J. (2021). Small data? No problem! Exploring the viability of pretrained multilingual language models for low-resourced languages. *Proceedings of the 1st Workshop on Multilingual Representation Learning*. https://aclanthology.org/2021.mrl-1.11

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., et al. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems, 32*.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825-2830.

Salton, G., and Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing and Management, 24*(5), 513-523.

Schuster, M., and Paliwal, K. K. (1997). Bidirectional recurrent neural networks. *IEEE Transactions on Signal Processing, 45*(11), 2673-2681.

Streamlit Inc. (2024). Streamlit [Software]. https://streamlit.io

Wang, S., and Manning, C. D. (2012). Baselines and bigrams: Simple, good sentiment and topic classification. *Proceedings of ACL 2012 (Short Papers)*. https://aclanthology.org/P12-2018

Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., et al. (2020). Transformers: State-of-the-art natural language processing. *Proceedings of EMNLP 2020: System Demonstrations*.